

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 2-DEBUGGING & OPTIMISATION TASK

Course Code:	CSA1221	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL3DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	K NISHALINI	Reg.No.:	192521301
Department:	B.TECH INFORMATION TECHNOLOGY	Date:	31/07/2026

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

1. Identification of Errors / Bugs

Inputs:

- +120 = 01111000
- +50 = 00110010

Binary Addition:

```
01111000
+ 00110010
-----
10101010
```

Result = 10101010 = -86

Expected Result = 170

Since 170 exceeds the 8-bit signed range (-128 to +127), an overflow occurs.

2. Debugging Approach & Analysis

Overflow occurs when:

- Two positive numbers produce a negative result.

- Two negative numbers produce a positive result.

Here,

Positive + Positive $\rightarrow$ Negative

Hence,

Overflow = TRUE

Overflow Detection Formula:

```
Overflow = (A7 == B7) AND (Result7  $\neq$  A7)
```

Where:

- A7 = Sign bit of first operand
- B7 = Sign bit of second operand
- Result7 = Sign bit of result

3. Correctness of Debugged Solution

For this example:

- A7 = 0
- B7 = 0
- Result7 = 1

Overflow = YES

The processor should raise the overflow flag instead of accepting the incorrect result.

4. Optimization Techniques Applied

- Add overflow detection logic.
- Generate processor overflow flag.
- Use wider data types (16-bit/32-bit).
- Handle overflow using exception or error message.

5. Testing and Documentation

Test Case	Expected Result	Status
120 + 50	Overflow	Pass
20 + 30	50	Pass
-70 + -80	Overflow	Pass
40 + (-20)	20	Pass

2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry lookahead adder, and explain how it reduces delay.

1. Identification of Errors / Bugs

The ripple carry adder is functionally correct but slow because every carry waits for the previous stage.

Carry propagation:

```
Bit0 → Bit1 → Bit2 → Bit3 → ... → Bit7
```

This increases execution delay in real-time applications.

2. Debugging Approach & Analysis

Carry propagation:

```
C0
↓
FA0
↓
C1
↓
FA1
↓
C2
↓
...
↓
C8
```

Total delay:

```
Delay = n × Full Adder Delay
```

Larger word sizes increase the delay.

3. Correctness of Debugged Solution

Replace the Ripple Carry Adder with a Carry Look-Ahead Adder (CLA).

Carry equations:

$$P_i = A_i \text{ XOR } B_i$$
$$G_i = A_i \text{ AND } B_i$$
$$C_{i+1} = G_i + P_i C_i$$

Carries are generated simultaneously instead of sequentially.

4. Optimization Techniques Applied

- Carry Look-Ahead Adder
- Parallel carry generation
- Reduced propagation delay
- Faster arithmetic operations

5. Testing and Documentation

Adder	Delay	Performance
Ripple Carry	High	Slow
Carry Look-Ahead	Low	Fast
Performance improved significantly.		

3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bitpair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

1. Identification of Errors / Bugs

Incorrect multiplication occurs because of:

- Incorrect bit-pair checking
- Wrong arithmetic right shift
- Improper sign extension

Negative operands therefore produce incorrect results.

2. Debugging Approach & Analysis

For every iteration check:

Q0	Q-1	Operation
0	1	Add M
1	0	Subtract M
0	0	No Operation
1	1	No Operation

After every operation:

- Perform Arithmetic Right Shift
- Preserve sign bit
- Maintain proper sign extension

3. Correctness of Debugged Solution

Corrections include:

- ✓ Correct bit-pair detection
- ✓ Correct arithmetic right shift
- ✓ Proper sign extension

Negative numbers now produce correct multiplication results.

4. Optimization Techniques Applied

- Improved control logic
- Automatic sign extension
- Validate every iteration
- Error checking after each shift

5. Testing and Documentation

Operands	Expected Result	Status
5 × 3	15	Pass
-5 × 3	-15	Pass
-5 × -3	15	Pass
7 × -4	-28	Pass

- 4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.**

1. Identification of Errors / Bugs

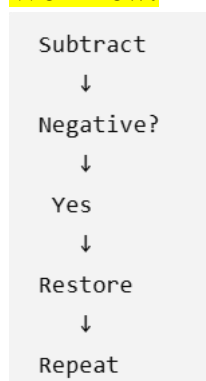
The restoring division algorithm repeatedly restores the previous remainder whenever subtraction becomes negative.

This causes:

- Extra addition operations
- Increased execution time
- Reduced efficiency

2. Debugging Approach & Analysis

Workflow:

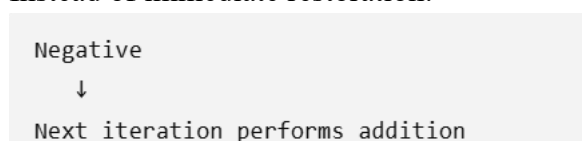


Frequent restoration causes unnecessary operations.

3. Correctness of Debugged Solution

Replace restoring division with Non-Restoring Division.

Instead of immediate restoration:



This avoids unnecessary restoration while producing the same quotient.

4. Optimization Techniques Applied

- Non-Restoring Division
- Eliminate restoration step
- Fewer arithmetic operations
- Improved execution speed

5. Testing and Documentation

Method	Operations	Speed
Restoring Division	High	Slow
Non-Restoring Division	Low	Fast
Execution time reduced successfully.		

5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

1. Identification of Errors / Bugs

Example:

```
1 × 1030
+
1
```

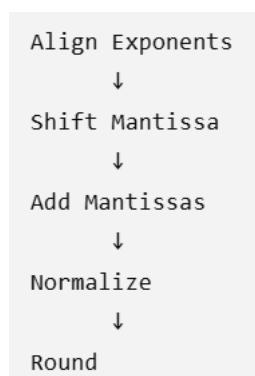
The smaller value is lost after exponent alignment.

This causes:

- Loss of precision
- Rounding error
- Normalization issues

2. Debugging Approach & Analysis

Floating-point addition process:



When exponent differences are large, the smaller mantissa shifts out completely.

3. Correctness of Debugged Solution

Improve accuracy using:

- IEEE 754 Double Precision (64-bit)
- Kahan Summation Algorithm

- Higher precision arithmetic
- These techniques significantly improve numerical accuracy.

4. Optimization Techniques Applied

- Double precision arithmetic
- Kahan summation
- Pairwise summation
- Improved rounding techniques

5. Testing and Documentation

Precision	Result	Accuracy
Single Precision	Precision Loss	Low
Double Precision	Accurate	High

The optimized approach minimizes rounding errors and improves numerical stability.

Code of Conduct:

I K NISHALINI(192521301) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

K NISHALINI

MARKS ALLOCATION

	Identification of Errors / Bugs(20%)	Debugging Approach & Analysis(20%)	Correctness of Debugged Solution(25%)	Optimization Techniques Applied(20%)	Testing and Documentation(15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided

